

# Advanced Data Handling Exercises

---

*8 activities: generators, itertools, dataclasses, safe serialization, chunked files, vectorized pandas, and memory profiling*

## EXERCISE 1

# Lazy Pipelines

---

- The baseline loader builds a full results list before returning anything - every item sits in memory at once, even for a dataset with a million rows.
- Replacing return with yield turns the function into a generator: each processed value is produced one at a time, on demand.
- Calling the generator function does not run any code yet - it only starts executing when something iterates it (here, `list(lazy_gen)`).
- The filtering condition (`item > 0`) and the transformation (`item * 10`) happen lazily, per element, instead of in one eager pass.

## Lazy Pipelines

ex1\_lazy\_pipeline.py

```
def load_all_records_baseline(data_list):
    results = []
    for item in data_list:
        if item > 0:
            results.append(item * 10)
    return results # everything materialised before returning

def stream_records_lazy(data_list):
    # yield turns this into a generator: nothing runs until iterated
    for item in data_list:
        if item > 0:
            yield item * 10 # one value produced at a time

raw_data = [5, -2, 12, 0, 8, -1]
print(load_all_records_baseline(raw_data))
lazy_gen = stream_records_lazy(raw_data)
print(hasattr(lazy_gen, "__next__")) # True: it's an iterator
print(list(lazy_gen)) # consuming it runs the generator body
```

### VERIFIED OUTPUT

```
[50, 120, 80] | Is generator: True | [50, 120, 80]
```

## EXERCISE 2

# Streaming with itertools

---

- The baseline groups records with `groupby`, but then calls `list(group)` to slice the first three items - materialising every group in full just to take a few elements.
- `itertools.islice(group, 3)` takes the first three items directly from the group iterator, without ever building the full list.
- `groupby` only groups consecutive items, so the data must be sorted by the grouping key first - both versions do this identically.
- Both versions must agree exactly: the `itertools` refactor changes how memory is used, not what the answer is.

## Streaming with itertools

ex2\_itertools\_pipeline.py

```
from itertools import islice, groupby

def process_data_pipeline_itertools(raw_records):
    # groupby only groups *consecutive* items, so sort by key first
    sorted_records = sorted(raw_records, key=lambda x: x["category"])
    grouped_totals = {}
    for key, group in groupby(sorted_records, key=lambda x: x["category"]):
        # islice pulls the first 3 items directly from the group
        # iterator -- no intermediate list is ever built
        first_three = islice(group, 3)
        grouped_totals[key] = sum(item["val"] for item in first_three)
    return grouped_totals

records = [
    {"category": "A", "val": 10}, {"category": "A", "val": 20},
    {"category": "A", "val": 30}, {"category": "A", "val": 40},
    {"category": "B", "val": 5}, {"category": "B", "val": 15},
    {"category": "B", "val": 25}, {"category": "B", "val": 35},
]
print(process_data_pipeline_itertools(records))
```

### VERIFIED OUTPUT

```
{'A': 60, 'B': 45} (matches the list-slicing baseline exactly)
```

### EXERCISE 3

## Validated Dataclasses

---

- `__post_init__` runs automatically right after a dataclass's generated `__init__` finishes — it is the natural place to validate fields.
- Raising `ValueError` inside `__post_init__` stops the object from being created at all: an invalid `OrderRecord` never exists, rather than existing in a broken state.
- Three independent guards check price, quantity, and `customer_id` - each raises its own clear message naming exactly what failed.
- This turns "a dict with the right keys" into "a value that is guaranteed valid the moment it exists."

## Validated Dataclasses

ex3\_order\_validation.py

```
from dataclasses import dataclass

@dataclass
class OrderRecord:
    order_id: str
    price: float
    quantity: int
    customer_id: str

    def __post_init__(self):
        # runs automatically right after the generated __init__
        if self.price <= 0:
            raise ValueError("price must be positive.")
        if self.quantity <= 0:
            raise ValueError("quantity must be positive.")
        if not self.customer_id:
            raise ValueError("customer_id must not be empty.")

valid = OrderRecord("ORD_001", 15.99, 3, "CUST_99")
print(f"Order created successfully: {valid}")

try:
    invalid = OrderRecord("ORD_002", -5.00, 2, "CUST_99")
except ValueError as e:
    print(f"Validation caught error: {e}")
```

### VERIFIED OUTPUT

Order created successfully: OrderRecord(...) | Validation caught error: price must be positive.

## EXERCISE 4

# Custom JSON Encoding

---

- `json.dumps()` has no idea how to serialise a dataclass instance by default - it only understands plain dicts, lists, strings, and numbers.
- Subclassing `json.JSONEncoder` and overriding `default()` lets you intercept exactly the objects the built-in encoder doesn't recognise.
- `dataclasses.asdict()` converts a dataclass instance into a plain dict, which the encoder can then serialise normally.
- Calling `super().default(obj)` for anything that isn't a dataclass preserves the built-in encoder's normal error behaviour for genuinely unsupported types.



## Custom JSON Encoding

ex4\_safe\_json\_encoder.py

```
import json
from dataclasses import dataclass, asdict, is_dataclass

@dataclass
class OrderRecord:
    order_id: str
    price: float
    quantity: int

class OrderEncoder(json.JSONEncoder):
    def default(self, obj):
        if is_dataclass(obj):
            return asdict(obj) # convert to a plain, serialisable dict
        return super().default(obj) # preserve normal error for others

order = OrderRecord("ORD_101", 45.50, 10)
payload = json.dumps(order, cls=OrderEncoder)
print(f"Encoded JSON: {payload}")

decoded_dict = json.loads(payload)
restored = OrderRecord(**decoded_dict) # rebuild the dataclass instance
print(f"Restored Record Object: {restored}")
```

### VERIFIED OUTPUT

```
{"order_id": "ORD_101", "price": 45.5, "quantity": 10} | Restored: OrderRecord(...)
```

## EXERCISE 5

# Chunked File Streaming

---

- `csv.DictReader` already reads one row at a time from disk - wrapping it in a generator (`yield row`) preserves that row-by-row behaviour all the way out to the caller.
- Opening the file with a `with` block guarantees the file handle closes automatically, even if the caller stops iterating early.
- The running totals dictionary is updated one row at a time as rows stream in - the full column of amounts is never held in memory together.
- This scales the same way whether the file has 4 rows or 4 million: memory use depends on the totals dict, not the file size.

## Chunked File Streaming

ex5\_csv\_stream.py

```
import csv

def stream_csv_records(file_path):
    # "with" guarantees the file closes even if iteration stops early
    with open(file_path, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            yield row # one row out at a time, never the whole file

totals = {}
for row in stream_csv_records("raw_data.csv"):
    cat = row["category"]
    amt = float(row["amount"])
    totals[cat] = totals.get(cat, 0.0) + amt # running total, no full column stored

print(f"Final aggregated totals: {totals}")
```

### VERIFIED OUTPUT

```
{'hardware': 500.0, 'software': 199.99}
```

## Vectorized pandas

---

- `df.iterrows()` loops through a DataFrame one Python-level row at a time - it is convenient, but it forfeits pandas' compiled, columnar speed entirely.
- `df["amount"] * 0.90` performs the same multiplication on the whole column at once, in compiled C code rather than a Python loop.
- `groupby("category").transform("sum")` computes each group's total and re-aligns it back onto every row of that group - no manual merge needed.
- Dividing one column by another (`net_amount / category_total`) computes every row's percentage share in a single vectorized expression.

## Vectorized pandas

ex6\_pandas\_vectorized.py

```
import pandas as pd

def calculate_metrics_vectorized(df):
    # column-wise arithmetic runs on the whole column at once
    df["net_amount"] = df["amount"] * 0.90
    # transform("sum") aligns each group's total back onto every row
    df["category_total"] = df.groupby("category")["net_amount"].transform("sum")
    # a column divided by a column: one vectorized expression, no loop
    df["percentage_share"] = df["net_amount"] / df["category_total"]
    return df

df_data = pd.DataFrame({
    "category": ["A", "A", "B", "B", "C"],
    "amount": [100.0, 200.0, 150.0, 50.0, 300.0],
})
print(calculate_metrics_vectorized(df_data))
```

### VERIFIED OUTPUT

category\_total: A=270.0, B=180.0, C=270.0 | percentage\_share sums to 1.0 within each category

## EXERCISE 7

# Memory Efficiency

---

- pandas defaults are generous: whole numbers become int64, decimals become float64, and repeated strings become the generic, memory-heavy object dtype.
- `astype("int16")` keeps the same whole numbers in a quarter of the space - fine as long as the values fit comfortably within int16's range.
- `astype("category")` is the biggest win for repeated strings: pandas stores each unique string once and reuses a small integer code for every repeat.
- `memory_usage(deep=True)` is essential for measuring object-dtype columns honestly - without `deep=True`, string columns are badly under-reported.

## Memory Efficiency

ex7\_memory\_optimization.py

```
import pandas as pd

def optimize_dataframe_dtypes(df):
    # repeated strings: store each unique value once, reuse a small code
    df["region_label"] = df["region_label"].astype("category")
    # whole numbers that comfortably fit in a smaller range
    df["status_code"] = df["status_code"].astype("int16")
    # decimals that don't need float64's extra precision
    df["temperature"] = df["temperature"].astype("float32")
    return df

df_raw = create_unoptimized_df()
mem_before = df_raw.memory_usage(deep=True).sum() # deep=True: honest string sizing
df_opt = optimize_dataframe_dtypes(df_raw.copy())
mem_after = df_opt.memory_usage(deep=True).sum()
savings = (mem_before - mem_after) / mem_before * 100
print(f"Before: {mem_before/1024:.2f} KB, After: {mem_after/1024:.2f} KB, Savings: {savings:.2f}%")
```

### VERIFIED OUTPUT

Before: 289.19 KB → After: 27.70 KB → Savings: 90.42%

## End-to-End Pipeline

---

- This capstone chains together three ideas from the whole workbook: a lazy generator (Exercise 1/5), dataclass validation (Exercise 3), and compact pandas dtypes (Exercise 7).
- Stage 1, `lazy_csv_stream`, yields raw dictionary rows from disk one at a time - nothing is loaded in bulk.
- Stage 2, `validate_and_filter_stream`, wraps each raw row in a Transaction dataclass; any row that fails validation (a negative amount) raises inside `__post_init__` and is quietly skipped.
- Because each stage is a generator, the whole pipeline runs one row through all three stages before moving to the next - not one stage finishing fully before the next begins.



# End-to-End Pipeline

ex8\_capstone\_pipeline.py

```
import csv
import pandas as pd
from dataclasses import dataclass

@dataclass
class Transaction:
    id_code: str
    category: str
    amount: float

    def __post_init__(self):
        if self.amount < 0:
            raise ValueError("Amount cannot be negative.")

def lazy_csv_stream(file_path):
    with open(file_path, newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            yield row # Stage 1: raw rows, one at a time

def validate_and_filter_stream(records_iterator):
    for row in records_iterator:
        try:
            yield Transaction(row["id_code"], row["category"], float(row["amount"]))
        except ValueError:
            continue # Stage 2: invalid rows skipped quietly

def aggregate_and_load_pandas(validated_iterator):
    categories, amounts = [], []
    for txn in validated_iterator: # Stage 3: consume the validated stream
        categories.append(txn.category)
        amounts.append(txn.amount)
    return pd.DataFrame({
        "category": pd.Series(categories, dtype="category"),
        "amount": pd.Series(amounts, dtype="float32"),
    })
```

## VERIFIED OUTPUT

DataFrame shape: (5000, 2) | 10 negative-amount rows correctly filtered out of 5010 total

# Key Takeaways

---

- `yield` turns a function into a generator: values are produced lazily, one at a time, instead of all at once.
- `itertools` tools like `islice` and `groupby` operate on iterators directly, avoiding intermediate lists entirely.
- `__post_init__` lets a dataclass validate itself the moment it is created, so invalid objects never exist.
- A custom `json.JSONEncoder.default()` is the standard way to teach `json.dumps()` how to serialise a type it doesn't know.
- Vectorized pandas operations and `groupby.transform()` replace row-by-row loops with fast, columnar expressions.
- Compact dtypes (`int16`, `float32`, `category`) and `memory_usage(deep=True)` turn memory optimisation into something you can measure, not guess at.